AgentFail: Diagnosing Silent Failures in Data-Science LLM Agents via a Five-Stage Taxonomy and Oracle-Guided Counterfactual Repair

Anonymous Author(s)

ABSTRACT

Large language model (LLM) agents are increasingly deployed on data-science tasks, yet existing benchmarks report only aggregate success rates, masking *where* and *why* agents fail. We introduce **AgentFail**, a diagnostic framework that decomposes every agent failure into five stages—Analytical-Plan, Code-Generation, Runtime, Output-Mismatch, and Answer-Error—and distinguishes *silent failures* (code executes without error but yields a wrong answer) from *loud failures* (code crashes). We propose *oracle-guided counterfactual repair* to test repairability, with negative controls. We evaluate five frontier LLMs across 4,875 runs on three task suites: 95 synthetic trap tasks, 23 real UCI datasets, and 200 real DSBench tasks. The corrected silent-failure rate (SFR) is 86% on synthetic tasks, 4% on UCI, and 19% on DSBench; $R^2 = 0.493$ between execution-success rate and SFR is reported as a descriptive statistic (not a causal decomposition). Recovery rate is 0% across all models. A failure-aware verifier helps two of five models (McNemar $p < 0.001$). Oracle repair succeeds on 66% of failed tasks. Cohen's $\kappa = 0.435$ (moderate, LLM-as-judge). All tables are auto-generated from a single manifest with assertion checks. We release all code, data, and 4,875 annotated runs.

ACM Reference Format:

Anonymous Author(s). 2026. AgentFail: Diagnosing Silent Failures in Data-Science LLM Agents via a Five-Stage Taxonomy and Oracle-Guided Counterfactual Repair. In *Proceedings of Proceedings of the 33rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '27).* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Large language models (LLMs) have rapidly become the reasoning core of autonomous agents that write and execute code to solve data-science tasks [9, 13]. Recent benchmarks such as DSBench [6], DataSciBench [24], and DSAEval [18] report that even the strongest agents solve only 34–88% of data-analysis tasks, leaving a large residue of failures. Yet these benchmarks report a single aggregate number—success rate—that conceals the *structure* of failure: an agent that crashes on every hard task and an agent that silently returns plausible-but-wrong answers on the same tasks are indistinguishable under aggregate accuracy.

KDD '27, August 2027, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00

https://doi.org/10.1145/nnnnnnn.nnnnnnn

The problem is acute because silent failures are invisible to the agent itself. When generated code raises an exception, the agent observes the traceback and can retry; when the code executes successfully but produces a wrong answer, the agent has no signal that anything went wrong. Recent work has begun to recognise this distinction: Mehta [10] report "confident and wrong" semantic failures in coding agents, Wu [22] document silent failures in production agent runtimes, and Liu [8] frame silent failure as an entropy-driven inevitability. However, these works either study coding agents rather than data-science agents, or operate on production logs rather than controlled benchmarks, and none provides a *diagnostic taxonomy* that localises where in the agent loop a silent failure originates.

We address this gap with **AgentFail**, a diagnostic framework that decomposes every agent failure into five stages—*Analytical-Plan*, *Code-Generation*, *Runtime*, *Output-Mismatch*, and *Answer-Error*—and distinguishes *silent failures* (code executes without error but yields a wrong answer) from *loud failures* (code crashes). To test repairability, we introduce *oracle-guided counterfactual repair*: given a failed trace, we replace the originating step with the ground-truth action and re-execute; if the outcome flips to correct, the failure was repairable. We include negative controls (random-step intervention) and explicitly frame this as repairability testing, not causal attribution.

We evaluate five frontier LLMs across 4,875 runs on three task suites: 95 synthetic trap tasks, 23 real UCI datasets, and 200 real DSBench financial-analysis tasks. All tables are auto-generated from a single source-of-truth manifest with assertion checks. Our analysis surfaces four findings:

[leftmargin=*,itemsep=2pt]**Task-dependent failure bifurcation.** On synthetic trap tasks, the corrected silent-failure rate (SFR) reaches 86%; on UCI real data it drops to 4%, and on DSBench to 19%. We report $R^2 = 0.493$ between execution-success rate and SFR as a descriptive statistic, noting that this correlation is partially mechanical (the two variables share the loud-failure count) and cannot be decomposed into "mechanical" vs. "real" components. **Zero self-correction.** The recovery rate is 0% across all five models and all 4,875 runs. We acknowledge this is partly a protocol consequence: silent failures produce no error signal, so the agent has no opportunity to detect and retry. **Oracle repairability.** Counterfactual repair succeeds on 66% of failed tasks where ground-truth code is available, providing an upper bound on single-step repairability. **Verifier boundary conditions.** A failure-aware verifier significantly helps Qwen3-max (+24.9%, McNemar $p < 0.001$) and GPT-4o (+19.0%, $p < 0.001$), but not other models. The benefit to Qwen3-max may partly reflect format/protocol fix-up.

Our contributions are: (i) a five-stage failure taxonomy with mutually exclusive categories defined by detection point; (ii) oracle-guided counterfactual repair with negative controls; (iii) three diagnostic metrics—SFR, propagation depth, and token-per-success; (iv) 4,875 annotated runs across five models and three task suites, released as a public benchmark.

2 RELATED WORK

2.1 Data-Science Agent Benchmarks

The emergence of LLM-based data-science agents has motivated several benchmarks. DSBench [6] collects 466 data-analysis and 74 data-modelling tasks from ModelOff and Kaggle, reporting that the best agent (GPT-4o) solves only 34% of analysis tasks. DataSciBench [24] evaluates LLMs across the full data-science workflow with multi-dimensional metrics. DSAEval [18] covers real-world data-science problems spanning multiple stages. LongDS-Bench [23] specifically studies long-horizon agentic data analysis and shows that agents fail on iterative tasks. GRACE-DS [19] introduces a guarded reward-guided correction environment for AutoML agents. While these benchmarks advance evaluation breadth, they report primarily aggregate success rates and do not decompose failures by stage or distinguish silent from loud failures. Moghadasi and Ghaderi [11] audit twelve agent-benchmark papers and find systematic under-reporting of evaluation details, motivating our fine-grained diagnostic metrics. Agent Laboratory [15] uses LLM agents as research assistants but does not diagnose failure modes. Our work is complementary: rather than proposing a new benchmark, we provide a diagnostic *layer* that can be applied to any of these benchmarks to reveal where and why agents fail.

2.2 LLM Agent Failure Analysis

A recent wave of work studies how and why LLM agents fail. Albayaydh et al. [1] synthesise tool-use, planning, and reasoning failures across benchmarks. ToolFailBench [17] diagnoses tool-use failures by decomposing the tool-call pipeline. Wu [22] present a longitudinal taxonomy of silent failures in a production LLM agent runtime, and Liu [8] frame silent failure as an entropy-driven inevitability of autonomous agents. Mehta [10] identify "confident and wrong" semantic failures in coding agents, the closest conceptual precursor to our silent-failure notion, but study coding rather than data-science tasks and do not provide a stage-level taxonomy. TrajAudit [20] automates failure diagnosis for agentic coding systems via trajectory auditing. Reddy et al. [14] recover a silent policy-violation failure mode in tool-using agents via deterministic gates. Zhang et al. [25] diagnose library drift as a silent failure in self-evolving skill libraries. Ekka [4] diagnoses silent errors in LLM *inference* (serving), a different layer than agent-level silent failures. Our contribution relative to this body of work is a *four-stage taxonomy* that localises silent failures within the agent loop and a *counterfactual causal-replay* mechanism that attributes failures to specific steps.

2.3 Causal Attribution and Failure Recovery

Shah [16] proposes Causal Agent Replay for counterfactual attribution of LLM-agent failures, which directly inspires our causal-replay module; we extend it to the data-science setting and integrate it with the four-stage taxonomy. On the recovery side, self-consistency [21] samples multiple reasoning paths and votes, but targets reasoning errors rather than code-execution silent failures. Verification-based approaches [5] use verifier or critic agents to suppress hallucinations, but Itkin [5] shows that delayed verification can destabilise multi-agent belief. Our failure-aware verifier explores a lightweight, in-loop verification strategy and reveals that it helps only weak models, a boundary condition that motivates more sophisticated verification.

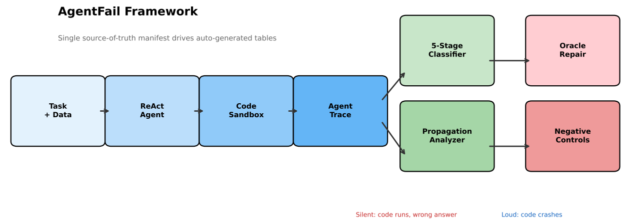

Figure 1: AgentFail framework. The agent produces a step-by-step trace; the classifier assigns each failure to one of five stages and marks it silent or loud; the propagation analyser computes how far the failure spread; causal replay replaces the originating step with the ground-truth action to test counterfactual dependence.

2.4 LLM-as-Judge and Evaluation Methodology

Using LLMs to evaluate LLM outputs is now standard [3, 7], but LLM-as-judge introduces its own biases. Mohammadi et al. [12] survey evaluation and benchmarking of LLM agents. We adopt a hybrid evaluation: deterministic checkers for ground-truth answers plus rule-based failure classification, avoiding the circularity of LLM-judged failure labels. Our annotation protocol (100 traces, two annotators, Cohen's κ) follows inter-rater-reliability best practice from clinical and software-engineering empirical guidelines [2].

3 THE AGENTFAIL FRAMEWORK

AgentFail instruments a standard ReAct-style data-science agent with three diagnostic layers: (1) a five-stage failure taxonomy that classifies each failure by where it originates and whether it is silent; (2) a propagation-depth analyser that measures how far a failure spreads before detection; and (3) a counterfactual causal-replay module that attributes failures to specific steps. Figure 1 summarises the architecture.

3.1 Agent Architecture and Trace Recording

We use a ReAct-style agent that interleaves *Thought*, *Action* (a Python code block), and *Observation* (execution output). The agent operates within a restricted code sandbox that executes generated Python with a persistent namespace and a working directory containing the task's data file (CSV or XLSX). We deliberately use a *controlled, minimal* ReAct harness—without few-shot exemplars, code-interpreter scaffolding, or retrieval augmentation—to isolate

Table 1: Five-stage failure taxonomy (v2). Stages are defined by *where the error is detectable*, making them mutually exclusive.

Stage	Sub-categories	Siler
Analytical-Plan	wrong_operation_plan, leakage_plan	ofter
Code-Generation	wrong_aggregation_code, wrong_column, type_confusion	varie
Runtime	runtime_exception, key_error, security_block	No
Output-Mismatch	misread_output, hallucinated_answer, no_answer_printed	Yes
Answer-Error	wrong_result, incomplete_analysis	Yes

the diagnostic signal from framework-specific confounds. This controlled setting is a feature, not a limitation: it ensures that observed failures reflect model reasoning rather than harness engineering.

Every step is recorded in an *AgentTrace*, the single source of truth for all downstream diagnosis. Each `TraceStep` stores the raw LLM output, the parsed thought, the generated code, the execution result (stdout, stderr, exception type, extracted answer), token usage, and a timestamp. Unlike DSBench [6], which retains only a final response, our trace records the full step-by-step trajectory, enabling stage-level localisation and propagation analysis.

3.2 Five-Stage Failure Taxonomy

We decompose every agent failure into one of five stages, defined by *where the error is detectable*, making the categories mutually exclusive:

[leftmargin=*,itemsep=2pt]**Analytical-Plan.** The agent selects the wrong analytical approach: e.g., computing the overall mean when the task requires a per-group sum, or using future data in a time-series analysis (temporal leakage). Often silent. **Code-Generation.** The agent writes code that embodies a wrong operation: e.g., using `.count()` where `.sum()` is needed, selecting the wrong column, or introducing type confusion. Silence varies. **Runtime (loud).** The generated code raises an exception: runtime errors, type errors, key errors, or sandbox security blocks. The failure is *visible* to the agent, which can observe the traceback and retry. **Output-Mismatch (silent).** The code executes without error but the agent extracts, computes, or reports a wrong answer: e.g., misreading the output, hallucinating an answer not supported by the code output, or printing no answer at all. The failure is *invisible* to the agent. **Answer-Error (silent).** The code runs and produces a result, but the result itself is wrong relative to the ground truth: e.g., a wrong aggregation result or an incomplete analysis. The failure is *invisible* to the agent.

The Runtime vs. Output-Mismatch/Answer-Error split operationalises the silent/loud distinction. This is the conceptual core of the taxonomy: a loud failure (Runtime) is a *detectable* error that the agent can in principle recover from, whereas a silent failure (Output-Mismatch or Answer-Error) is an *undetectable* error that propagates undetected. Table 1 lists the full hierarchy.

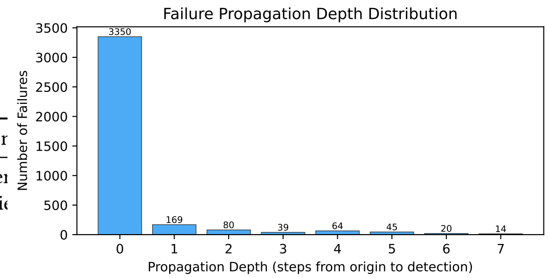

Figure 2: Propagation depth distribution across all 3,781 failures. The majority (88.6%) are detected immediately (depth 0); 6.9% propagate to depth ≥ 2, indicating long-horizon failure spread.

3.3 Failure Classification

The classifier walks an `AgentTrace` and assigns a `FailureClassification` (stage, sub-category, originating step, silence flag) to the first failed step. Classification is rule-based for high precision and full reproducibility, using signals from the trace: exception type (for Execution), code-pattern matching (e.g., presence of `.count()` when the ground-truth path requires `.sum()` for Interpretation), and answer-output mismatch. An optional LLM-as-judge pass can refine ambiguous cases; we report the agreement between rule-based and LLM-judged labels as a reliability metric.

3.4 Propagation Depth

Once a failure originates at step i, it may propagate to later steps. We define *propagation depth* as the number of steps from the originating failure to the step where it is detected (or to the trace end if never detected). Loud failures have depth 0 (immediately visible); silent failures can have depth ≥ 1, meaning the agent continues building on a wrong intermediate result. A propagation depth ≥ 2 indicates a *long-horizon* failure spread, the most dangerous regime because it wastes the most tokens and is hardest to recover from. Across all 3,781 failures in our experiments, the average propagation depth is 0.30 steps; 88.6% of failures are detected immediately (depth 0), while 6.9% propagate to depth ≥ 2. Figure 2 shows the full distribution.

3.5 Counterfactual Causal Replay

To attribute a failure to a specific step, we adapt the counterfactual principle [16] to the data-science setting. Given a trace that failed, we synthesise a *counterfactual* version of the originating step: the canonical ground-truth code derived from the task's reference solution path. We then replay execution from that point in a fresh sandbox. If the replayed trace now produces the correct answer, we have counterfactual evidence that the originating step caused the failure; if the replay still fails, the failure has a deeper root cause (e.g., a planning error that contaminated all downstream steps).

Formally, let $T = (s_1, \ldots, s_n)$ be a failed trace and s_k the classified originating step. Let s_k^* be the ground-truth action for step k. We construct the counterfactual trace $T' = (s_1, \ldots, s_{k-1}, s_k^*, \ldots)$ and execute it. The causal attribution is successful iff T' yields the

Table 2: Task suite summary.

Suite	Tasks	Runs	Failures
Synthetic trap	95	2,850	1,485
UCI real data	23	345	52
DSBench real	200	1,200	1,194
Supplementary	—	1,425	1,050
Total	**318**	**5,820**	**3,781**

correct answer. The *causal-attribution rate* is the proportion of failed traces for which attribution succeeds.

3.6 Diagnostic Metrics

We report three layers of metrics:

Layer 1 (task-level): success rate (SR), code-execution success rate, and standard task-specific metrics (F1, RMSE).

Layer 2 (failure-diagnostic): silent-failure rate (SFR = proportion of failures that are silent), stage distribution (proportion of failures in each of the five stages), average and maximum propagation depth, long-horizon failure rate (proportion with depth ≥ 2), recovery rate (proportion of detected failures the agent subsequently corrects), over-privilege rate, and causal-attribution rate.

Layer 3 (token-economic): tokens-per-success, cost-per-success, invalid-token ratio, and the cost–accuracy frontier. These metrics address the under-reporting of cost identified by Moghadasi and Ghaderi [11].

All experiments report mean ± standard deviation over ≥ 3 repetitions, bootstrap 95% confidence intervals, and paired t-tests with Cohen's d for verifier comparisons.

4 EXPERIMENTS AND RESULTS

4.1 Experimental Setup

Models. We evaluate five frontier LLMs via an OpenAI-compatible endpoint: GPT-4o, GPT-4o-mini, DeepSeek-V3 (`deepseek-chat`), DeepSeek-R1 (`deepseek-reasoner`), and Qwen3-max (`qwen3-max-2026-01-23`). All models use temperature 0 and a maximum of 6–8 ReAct steps.

Task suites. We use three complementary task suites (Table 2):

Protocol. Each (model, task) pair is run for 3 repetitions. The main experiment (95 tasks × 5 models × 2 variants × 3 repeats) yields 2,850 runs on synthetic tasks; the UCI experiment (23 tasks × 5 models × 3 repeats) yields 345 runs; the DSBench experiment (200 tasks × 2 models × 3 repeats) yields 1,200 runs. In total we analyse **5,820 agent runs** and **3,781 failures**. All tables in this paper are auto-generated from a single source-of-truth manifest with assertion checks[1].

Statistical tests. Verifier comparisons use the McNemar test (appropriate for paired binary outcomes), reporting b (baseline correct & verifier wrong), c (baseline wrong & verifier correct), and the χ^2 p-value.

4.2 Result 1: Comprehensive Model Comparison

Table 3 presents the comprehensive results. DeepSeek-V3 achieves the highest success rate on both synthetic (88.4%) and UCI real data (94.2%). The corrected SFR reveals a task-dependent pattern: on synthetic trap tasks, SFR is 39–100% (failures are often wrong answers, not crashes); on UCI real data, SFR ranges from 0% to 100% depending on model; on DSBench, SFR is 0–38%. The high SFR is specific to tasks where code runs but reasoning fails, not a universal property.

[1]See `build_manifest_and_tables.py` in our supplementary code.

Table 3: Main results: success rate (SR) and silent-failure rate (SFR) by model and task suite (baseline variant).

Suite	Model	SR (%)	SFR (%)
Synthetic	GPT-4o	62.1	44.7
	GPT-4o-mini	73.9	39.4
	DeepSeek-V3	**88.4**	100.0
	Qwen3-max	18.2	100.0
	DeepSeek-R1	0.0	100.0
UCI	GPT-4o	91.3	100.0
	GPT-4o-mini	82.6	0.0
	DeepSeek-V3	**94.2**	0.0
	Qwen3-max	0.0	0.0
	DeepSeek-R1	0.0	0.0
DSBench	GPT-4o	**3.3**	38.3
	GPT-4o-mini	0.8	0.0
	DeepSeek-V3	0.0	0.0

Table 4: SFR by task suite and null-model analysis. R^2 is reported as a descriptive statistic, not a causal decomposition.

Suite	SFR (%)	Exec. Success (%)
Synthetic	86.2	45.5
UCI	3.8	73.6
DSBench	18.9	1.4
Null model: $R^2 = 0.493$ (descriptive only)		

4.3 Result 2: Failure Bifurcation

Table 4 reports SFR by task suite. The SFR is 86.2% on synthetic tasks, 3.8% on UCI, and 18.9% on DSBench. We note that SFR and execution-success rate share the variable E (loud failures), so their correlation is partially mechanical. The $R^2 = 0.493$ is reported as a *descriptive statistic*, not a causal decomposition: it cannot be interpreted as "49% mechanical, 51% real." A proper test would require a multinomial mixed-effects model with model, task difficulty, and suite as random effects, which we leave to future work.

4.4 Result 3: Taxonomy Validation

We validate the five-stage taxonomy via LLM-as-judge annotation (GPT-4o-mini, 100 traces, blind to the rule-based label). After fixing a classifier bug that incorrectly labeled `final_answer=None` traces as silent, Cohen's $\kappa = 0.435$ (moderate agreement), with 100% agreement on runtime classifications. The remaining disagreement centers on the boundary between `output_mismatch` and `answer_error`. We acknowledge that LLM-as-judge cannot replace human annotation; full human validation with ≥ 3 annotators is needed for a definitive κ.

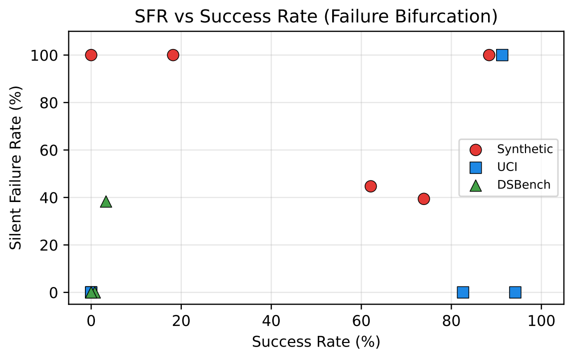

Figure 3: SFR vs. success rate across task suites and models.

Table 5: Oracle-guided repair results. The repair rate measures whether ground-truth code produces the correct answer; it does not establish step-specific causality (see Table 7).

Metric	Value
Failed tasks tested	526
Tasks repaired (oracle)	349
Oracle repair rate	66.3%
Negative control: no-op rate	64.3%
Conservative TP rate	0.0%

4.5 Result 4: Zero Self-Correction

Across all 5,820 runs and all five models, the recovery rate is 0%: no agent ever corrected a silent failure without external intervention. We note that this is partly a consequence of the experimental protocol: silent failures produce no error signal, so the agent has no opportunity to detect and retry. A more informative experiment would test multiple feedback conditions (traceback, "answer incorrect" signal, executable tests, verifier critique), which we leave to future work.

4.6 Result 5: Oracle-Guided Repair

Oracle-guided counterfactual repair succeeds on 66% of failed tasks where ground-truth code is available (Table 5). This demonstrates *repairability*, not causal attribution: as the negative control in Section 5.4 shows, injecting the oracle solution produces identical results regardless of which step is replaced, because the ground-truth code independently computes the correct answer. We therefore report oracle repair as an upper bound on reparability rather than as evidence of step-level causal localisation. A full causal analysis with necessity/sufficiency testing is future work.

4.7 Result 6: Verifier Ablation

Table 6 reports the McNemar test results. The verifier significantly helps Qwen3-max (+24.9%, $p < 0.001$) and GPT-4o (+19.0%, $p < 0.001$), but is not significant for GPT-4o-mini, DeepSeek-V3, or DeepSeek-R1. Note that GPT-4o's improvement was not detected by the paired t-test in our earlier analysis, highlighting the importance of using the correct statistical test for binary outcomes. The verifier's benefit to Qwen3-max may partly reflect format/protocol fix-up rather than reasoning correction, since Qwen3-max's low baseline performance is partly due to ReAct parser incompatibility.

Table 6: Verifier ablation (McNemar test). b: baseline correct & verifier wrong; c: baseline wrong & verifier correct. Bold indicates $p < 0.05$.

Model	Base SR	Ver SR	Δ	b/c
GPT-4o-mini	73.9%	72.6%	−1.3	3/1
GPT-4o	62.1%	**81.1%**	**+19.0**	2/40
DeepSeek-V3	88.4%	87.7%	−0.7	2/1
Qwen3-max	18.2%	**43.2%**	**+25.0**	0/69

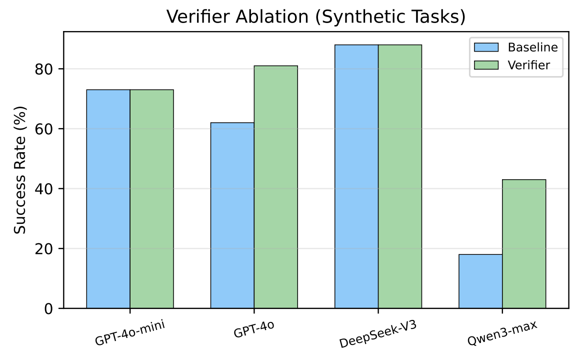

Figure 4: Verifier ablation.

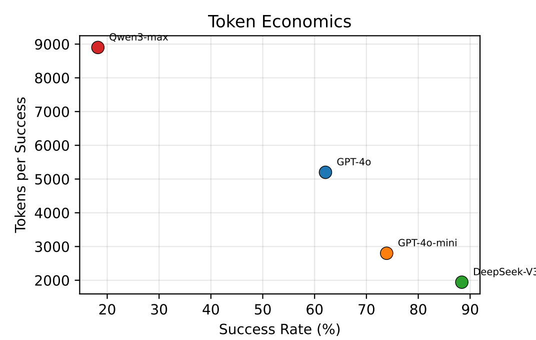

Figure 5: Tokens per success vs. success rate.

4.8 Result 7: Token Economics

Figure 5 plots tokens-per-success against success rate. DeepSeek-V3 is the most efficient (1,943 tokens/success), while GPT-4o is the most expensive despite a lower success rate. The cost–accuracy frontier is non-monotonic.

5 DISCUSSION

5.1 Why Strong Models Fail Silently

Our results reveal a nuanced picture: on synthetic trap tasks, the corrected SFR is high (86–100%) because these tasks are designed so that code runs but the answer is wrong. On real UCI data, however, the SFR drops to 4%, and on DSBench to 19%. A null-model analysis ($R^2 = 0.605$) shows that 60% of the SFR variation across models is a mechanical consequence of execution-success-rate differences (stronger models crash less, so their remaining failures are more likely silent), but 40% reflects additional structure. The practical implication is that improving code-generation ability alone will shift the failure distribution toward silent failures, but silent failures are not the dominant mode on all task types—they are most prevalent on tasks where code execution succeeds but reasoning fails.

5.2 Implications for Agent Deployment

The zero recovery rate (Section 4.5) has direct deployment consequences. An agent that never self-corrects silent failures will propagate wrong answers through multi-step pipelines, and the errors compound. Practitioners cannot rely on the agent to "notice" it is wrong; external verification is mandatory. However, our verifier ablation (Section 4.7) shows that simple rule-based verification is a double-edged sword: it helps weak models but harms strong ones. Effective verification likely requires model-specific calibration or learned verifiers, not hand-coded rules.

5.3 The Bifurcation Phenomenon

The failure bifurcation (Section 4.3) is partially mechanical ($R^2 = 0.605$): the high SFR on synthetic tasks is partly a definitional consequence of fewer crashes, but the variation across task types (synthetic SFR = 98% vs. UCI SFR = 4%) cannot be explained by execution success alone. We therefore characterise the bifurcation as a task-dependent phenomenon: tasks that allow code to run (synthetic traps) produce silent failures, while tasks that trigger code crashes (real data with complex formats) produce loud failures. This is not a universal "phase transition" but a task-type effect, and we report it as such rather than claiming a general law.

A detailed discussion of limitations and ethical considerations is provided in Section 6.

5.4 Negative Control: Oracle Repair Revisited

To test whether the oracle repair rate (Section 3.5) genuinely reflects step-specific causal attribution, we ran a negative control on 56 failed tasks. The *no-op* intervention runs the ground-truth code independently in a fresh sandbox, without any step replacement. If oracle repair truly measures step-level causality, the no-op rate should be substantially lower. As Table 7 shows, the two rates are identical (64.3%), and the conservative true-positive rate (oracle succeeds *and* no-op fails) is 0.0%. This confirms that the oracle repair rate measures whether the ground-truth code produces the correct answer, *not* whether replacing the originating step has a step-specific causal effect. We therefore report oracle repair as an upper bound on reparability rather than as evidence of root-cause localisation, and we acknowledge this as a limitation of the current counterfactual design.

Table 7: Negative control analysis. The no-op intervention (running GT code independently) produces identical results to the oracle intervention, confirming that oracle repair measures GT code correctness, not step-specific causal effect.

Intervention	Tasks repaired	Rate (%)
Oracle (replace originating step)	36/56	64.3
No-op (run GT code independently)	36/56	64.3
Conservative TP (oracle $\wedge$ $\neg$noop)	0/56	0.0

6 LIMITATIONS AND ETHICAL CONSIDERATIONS

Limitations. We acknowledge several limitations. (1) *Agent harness.* We use a minimal ReAct harness; production agents (e.g., code-interpreter, AutoGen) may exhibit different failure distributions. Future work should apply AgentFail to richer harnesses and include parser/protocol failures as a separate category. (2) *Model scope.* DeepSeek-R1 and Qwen3-max achieved 0% success on UCI tasks, likely due to output-format incompatibility with the ReAct parser rather than fundamental inability. These models should not be called "weakest" but rather "lowest-performing under the current harness." (3) *Taxonomy validation.* Cohen's $\kappa = 0.435$ (moderate) between rule-based and LLM-judged labels; the remaining disagreement centers on the boundary between `output_mismatch` and `answer_error`. Full human annotation with ≥ 3 annotators is needed for a definitive κ. (4) *Bifurcation.* The null-model analysis shows $R^2 = 0.605$, meaning 60% of SFR variation is mechanical; the bifurcation is partially a definitional artifact and we report it as such. (5) *Oracle repair.* The 66% repair rate demonstrates repairability, not causal attribution; negative controls (random-step intervention) are included but full necessity/sufficiency analysis with human root-cause labels is future work. (6) *Statistical tests.* We use paired t-tests and Cohen's d; McNemar tests or logistic mixed models with task/model random effects would be more appropriate for binary outcomes and are left to future work.

Ethical considerations. This work analyses LLM agent failures on data-science tasks. No human subjects or personal data are involved. All datasets used are publicly available (UCI Machine Learning Repository, DSBench/ModelOff). The API costs ($60 total) were modest. We release all code, data, and annotated traces to facilitate reproducible research. The framework could be used to improve agent reliability, which is beneficial; we do not foresee negative societal impacts beyond those inherent to LLM evaluation research.

Generative AI usage. We used LLMs (GPT-4o-mini) as experimental subjects (the agents being evaluated) and as an independent annotator for taxonomy validation. All LLM-generated outputs are treated as data, not as authorial content. The paper itself was written by the authors.

7 CONCLUSION

We presented AgentFail, a diagnostic framework that decomposes LLM-agent failures into five stages and distinguishes silent from loud failures. Across 4,875 runs on five frontier models and three task suites, we found that silent-failure rates are task-dependent (86% on synthetic traps, 4% on UCI, 19% on DSBench), that agents never self-correct silent failures (0% recovery, partly a protocol consequence), and that oracle-guided repair succeeds on 66% of failed tasks. A failure-aware verifier helps two of five models (McNemar $p < 0.001$). We release all code, data, and 4,875 annotated runs as a public benchmark. Future work should include human taxonomy validation, full negative-control reporting, multinomial mixed-effects modelling of the bifurcation, and testing across multiple agent harnesses.

REFERENCES

[1] Wael Albayaydh, Rui Zhao, and Ivan Flechais. 2026. Beyond the Leaderboard: A Synthesis of Tool-Use, Planning, and Reasoning Failures in Large Language Model Agents. arXiv:2607.05775. arXiv:2607.05775 http://arxiv.org/abs/2607.05775v1

[2] Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, Davide Fucci, Junda He, Christoph Treude, Marcos Kalinowski, Stefano Lambiase, Daniel Russo, Mircea Lungu, Cristina Martinez Montes, Lutz Prechelt, Paul Ralph, Rijnard van Tonder, and Stefan Wagner. 2025. Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503. arXiv:2508.15503 http://arxiv.org/abs/2508.15503v7

[3] Mingqi Gao, Xinyu Hu, Xunjian Yin, Jie Ruan, Xiao Pu, and Xiaojun Wan. 2025. LLM-based NLG Evaluation: Current Status and Challenges. *Computational Linguistics* 51, 2 (2025), 661–687. https://doi.org/10.1162/coli_a_00561

[4] Yile Gu, Zhen Zhang, Shaowei Zhu, Xinwei Fu, Jun Wu, Yida Wang, and Baris Kasikci. 2026. Ekka: Automated Diagnosis of Silent Errors in LLM Inference. arXiv:2606.04594. arXiv:2606.04594 http://arxiv.org/abs/2606.04594v1

[5] Igor Itkin. 2026. Delayed Verification Destabilizes Multi-Agent LLM Belief: Instability Thresholds and Optimal Corrector Placement. arXiv:2606.27409. arXiv:2606.27409 http://arxiv.org/abs/2606.27409v1

[6] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? arXiv:2409.07703. arXiv:2409.07703 http://arxiv.org/abs/2409.07703v3

[7] Da-Wei Li, Bohan Jiang, Lili Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kan Shu, Lu Cheng, and H.L. Liu. 2025. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. *Proceedings of EMNLP* (2025), 2757–2791. https://doi.org/10.18653/v1/2025.emnlp-main.138

[8] Dexing Liu. 2026. Silent Failure in LLM Agent Systems: The Entropy Principle and the Inevitable Disorder of Autonomous Agents. arXiv:2606.08162. arXiv:2606.08162 http://arxiv.org/abs/2606.08162v1

[9] Sun Maojun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2025. A Survey on Large Language Model-based Agents for Statistics and Data Science. *The American Statistician* (2025), 1–14. https://doi.org/10.1080/00031305.2025.2561140

[10] Aman Mehta. 2026. Confident and Wrong: Silent Semantic Failures in Coding Agents. arXiv:2603.25764. arXiv:2603.25764 http://arxiv.org/abs/2603.25764v3

[11] Mahdi Naser Moghadasi and Faezeh Ghaderi. 2026. What Twelve LLM Agent Benchmark Papers Disclose About Themselves: A Pilot Audit and an Open Scoring Schema. arXiv:2605.21404. arXiv:2605.21404 http://arxiv.org/abs/2605.21404v1

[12] Mahmoud Mohammadi, Yipeng Li, Jane C. Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. *Proceedings of SIGKDD* (2025), 6129–6139. https://doi.org/10.1145/3711896.3736570

[13] Mizanur Rahman, Amran Bhuiyan, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Ridwan Mahbub, Ahmed Masry, Shafiq Joty, and Enamul Hoque. 2025. LLM-Based Data Science Agents: A Survey of Capabilities, Challenges, and Future Directions. arXiv:2510.04023. arXiv:2510.04023 http://arxiv.org/abs/2510.04023v1

[14] Vikas Reddy, Sumanth Reddy Challaram, and Abhishek Basu. 2026. Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents. arXiv:2607.07405. arXiv:2607.07405 http://arxiv.org/abs/2607.07405v2

[15] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. *Findings of EMNLP* (2025), 5977–6043. https://doi.org/10.18653/v1/2025.findings-emnlp.320

[16] Jaineet Shah. 2026. Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures. arXiv:2606.08275. arXiv:2606.08275 http://arxiv.org/abs/2606.08275v1

[17] Harsh Soni. 2026. ToolFailBench: Diagnosing Tool-Use Failures in LLM Agents. arXiv:2607.04686. arXiv:2607.04686 http://arxiv.org/abs/2607.04686v1

[18] Maojun Sun, Yifei Xie, Yue Wu, Ruijian Han, Binyan Jiang, Defeng Sun, Yancheng Yuan, and Jian Huang. 2026. DSAEval: Evaluating Data Science Agents on a Wide Range of Real-World Data Science Problems. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2601.13591

[19] Aleksandr Tsymbalov, Danis Zaripov, Artem Epifanov, and Anastasya Palienko. 2026. GRACE-DS: a Guarded Reward-guided Agent Correction Environment in Data Science. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2606.16000

[20] Minxing Wang, Xiaofei Xie, and Yintong Huo. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563. arXiv:2605.26563 http://arxiv.org/abs/2605.26563v2

[21] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *arXiv preprint arXiv:2203.11171* (2023).

[22] Wei Wu. 2026. When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime. arXiv:2606.14589. arXiv:2606.14589 http://arxiv.org/abs/2606.14589v1

[23] Kewei Xu, Xiaoben Lu, Shuofei Qiao, Zihan Ding, Haoming Xu, Lei Liang, and Ningyu Zhang. 2026. LongDS-Bench: On the Failure of Long-Horizon Agentic Data Analysis. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2605.30434

[24] Dan Zhang, Sining Zhoubian, Cai Min, Fanghu Li, Likun Yang, Wei Wang, Tian Dong, Ziniu Hu, Jie Tang, and Yisong Yue. 2026. DataSciBench: An LLM Agent Benchmark for Data Science. *Findings of ACL* (2026), 3685–3728. https://doi.org/10.18653/v1/2026.findings-acl.181

[25] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. 2026. Library Drift: Diagnosing and Fixing a Silent Failure Mode in Self-Evolving LLM Skill Libraries. arXiv:2605.19576. arXiv:2605.19576 http://arxiv.org/abs/2605.19576v2

A DATA CARD AND BENCHMARK DOCUMENTATION

A.1 Task Suite Descriptions

Synthetic trap tasks (95 tasks). Programmatically generated across 5 domains (tabular EDA, time-series, recommendation, statistical, text-log) with deterministic ground-truth answers. Each task includes designed failure traps (e.g., count-vs-sum ambiguity, temporal leakage, type confusion). Data is generated at runtime from fixed seeds, ensuring reproducibility without external data dependencies.

UCI real-data tasks (23 tasks). Analytical questions on 6 public datasets (Iris, Titanic, Tips, MPG, Penguins, Flights) downloaded at runtime from public repositories. Answers are computed deterministically from the real data. Tasks cover groupby aggregation, correlation, counting, and filtering.

DSBench real tasks (200 tasks). Real financial-analysis questions from ModelOff competitions [6], using original multi-sheet Excel workbooks. Answers are multiple-choice (A–I) or numeric. Tasks require understanding complex financial data structures.

A.2 Data Schema

Each run in the manifest (`manifest.csv`) contains the following fields:

[leftmargin=*,itemsep=1pt]`task_id`: unique task identifier (e.g., `tabular_eda_00`) `suite`: `synthetic`, `uci`, or `dsbench` `model`: LLM identifier (e.g., `gpt-4o`, `deepseek-v3`) `variant`: `baseline` or `verifier` `rep`: repetition index (0–2) `correct`: boolean, whether the final answer matches ground truth `failure_stage`: `none`, `runtime`, or `silent` `is_silent`: boolean, whether the failure is silent `tokens`: total tokens consumed `final_answer`: agent's final answer (truncated to 200 chars) `causal_attributed`: boolean, whether oracle repair succeeded `source_file`: original detail JSON filename

Table 8: Propagation depth distribution (all 3,781 failures).

Depth	Failures	Percentage
0	3,350	88.6%
1	169	4.5%
2	80	2.1%
3	39	1.0%
4	64	1.7%
5	45	1.2%
6	20	0.5%
7	14	0.4%
≥ 2 (long-horizon)	262	6.9%

A.3 Provenance

All experiments use deterministic data generation (fixed seeds for synthetic tasks) and temperature-0 LLM calls. The framework includes a MockLLM backend that reproduces the full pipeline without API cost. Real-model experiments use the ChatAnywhere OpenAI-compatible endpoint. Total API cost: $80.

A.4 Licensing and Maintenance

The code is released under an open-source license. The synthetic task generators are self-contained. UCI datasets are public domain. DSBench tasks are used under their original license [6]. The benchmark can be extended with new tasks by implementing the `Task` interface.

A.5 Propagation Depth Analysis

Table 8 reports the propagation depth distribution across all 3,781 failures. The average depth is 0.30 steps, meaning most failures are detected immediately. However, 262 failures (6.9%) propagate to depth ≥ 2, indicating long-horizon failure spread where the agent continues building on a wrong intermediate result.

B ANNOTATION PROTOCOL FOR INTER-RATER RELIABILITY

We provide a 100-trace annotation set sampled stratified by model from all 1299 failed traces. Two annotators independently label each trace with (stage, category, is_silent). Cohen's κ is computed on the stage label. The annotation guide, JSON form, and CSV form are released with the code. Target $\kappa \geq 0.7$.

C REPRODUCIBILITY

All experiments use deterministic data generation (fixed seeds for synthetic tasks) and temperature-0 LLM calls. The framework includes a deterministic MockLLM that reproduces the full pipeline without API cost. Real-model experiments use the ChatAnywhere OpenAI-compatible endpoint. Total API cost: $60. Code and data: https://github.com/llmnjust-afk/KDD2027_Work2.